

Introduction Cloud systems: virtualization

ADAM BELLOUM(UVA)



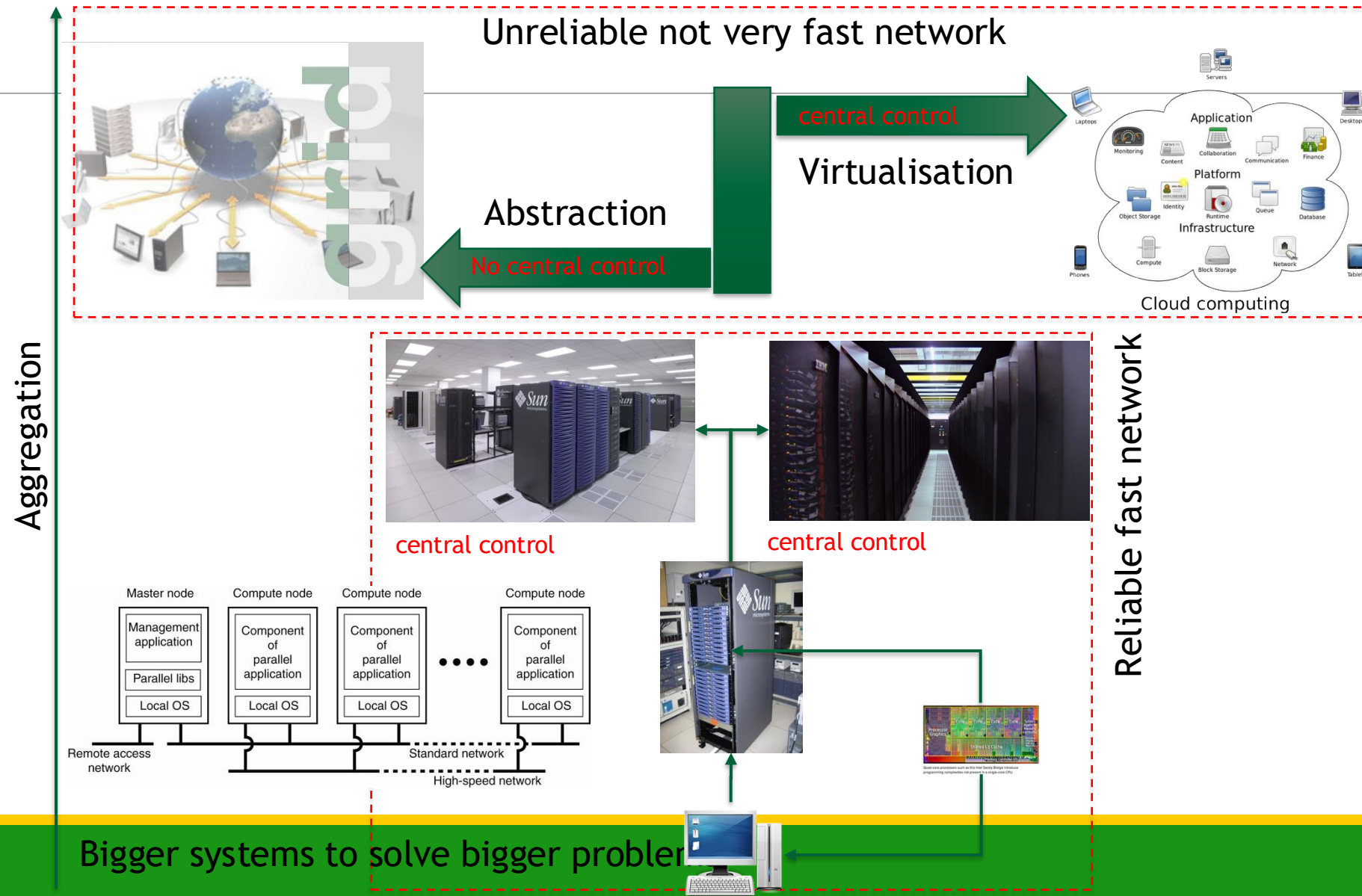
Funded by
the European Union

DIGITAfrica project has received funding from the EU Horizon Europe research and innovation Programme and Switzerland under Grant Agreement No. 101187966. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Executive Agency (REA). Neither the European Union nor the granting authority can be held responsible for them.





From mono-core to exa-scale computer

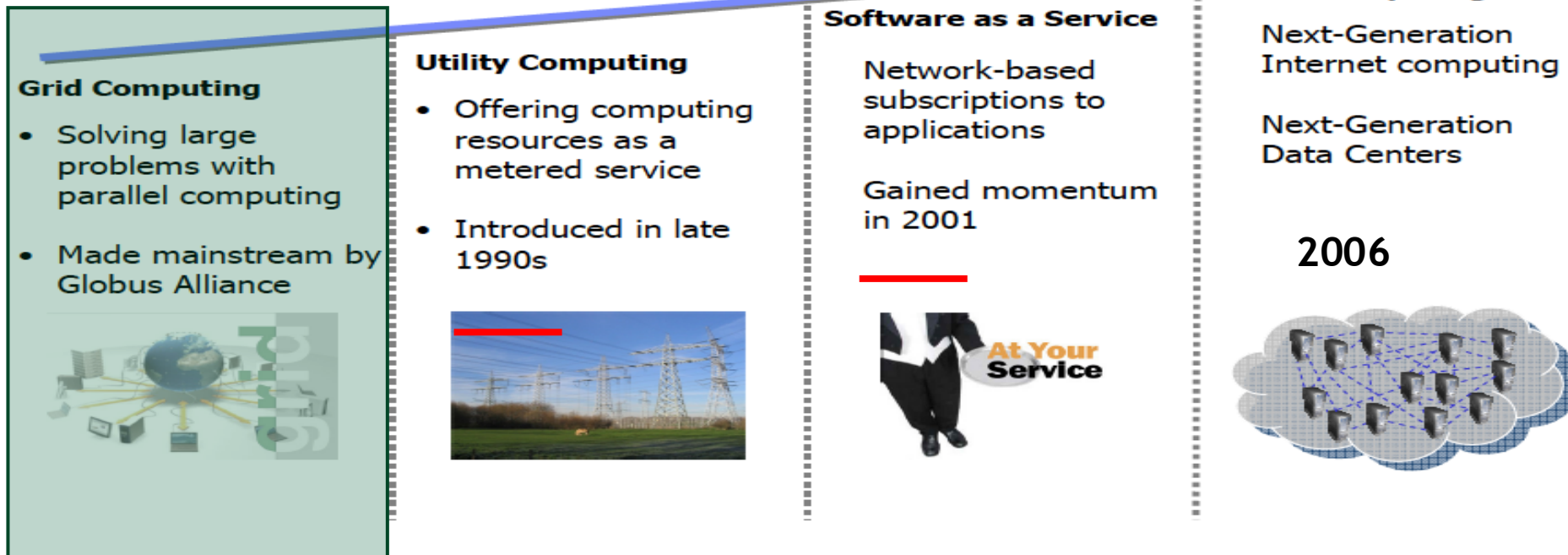




Evolution of Distributed systems



Cloud Computing is an Evolution in IT



Grid 1.0

Grid 2.0



What is Cloud computing



CLOUD COMPUTING IS A MODEL FOR ENABLING UBIQUITOUS, CONVENIENT, ON-DEMAND NETWORK ACCESS TO A SHARED POOL OF CONFIGURABLE COMPUTING RESOURCES

- RESOURCES:
 - NETWORKS,
 - SERVERS,
 - STORAGE,
 - APPLICATIONS / SERVICES

THAT CAN BE

RAPIDLY PROVISIONED/ RELEASED WITH MINIMAL MANAGEMENT EFFORT OR SERVICE PROVIDER INTERACTION.

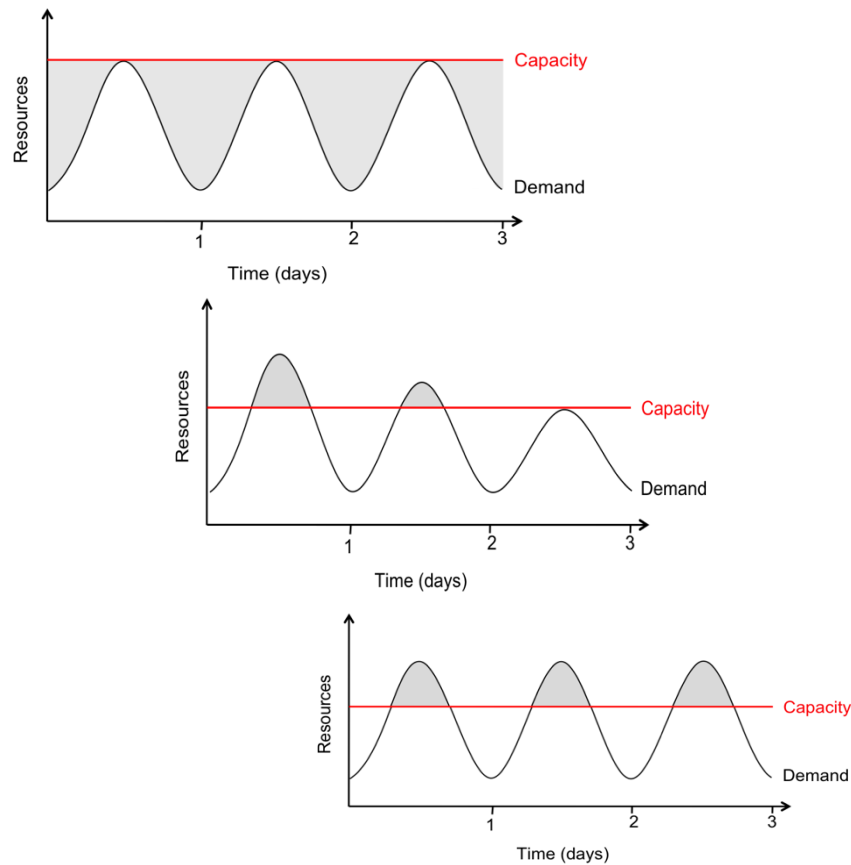
THIS CLOUD MODEL PROMOTES AVAILABILITY AND IS COMPOSED OF

- 5 ESSENTIAL CHARACTERISTICS,
- 3 SERVICE MODELS,
- AND 4 DEPLOYMENT MODELS.

- Five Cloud characteristics
 - On-demand self-service
 - Broad network access
 - Resource pooling
 - Rapid elasticity
 - Measured Service
- 3 service/provisioning models
 - Software as a Service (SaaS)
 - Platform as a Service (PaaS)
 - Infrastructure as a Service (IaaS)
- 4 deployment models
 - Private Cloud
 - Public Cloud
 - Hybrid Cloud
 - Community Cloud



The provisioning problem



Capacity vs Demand

Users



providers



Users



providers



Users



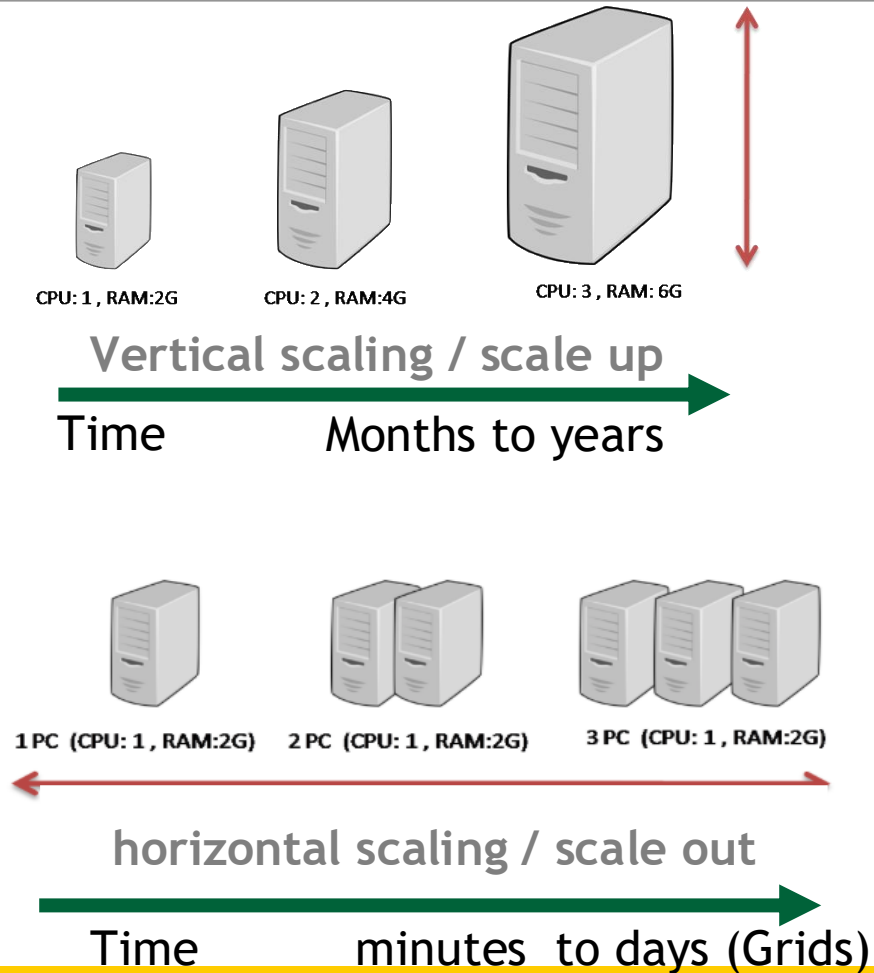
providers



Time



Elastic approach to resource provisioning



horizontal scaling / scale out / Elastic



Time Seconds to minutes (Clouds)



Style of computing & usage model

A style of computing where massive salable IT capabilities are provided:

- **as a Service** → (IaaS, PaaS, SaaS, <XaaS>, ...)
- over the network

Cloud characteristics

- Single point of Access
- Virtualization
- Automation
- Flexibility
- Usage accounting
- Service Management
- Security
- Cost efficiency

Usage model

- Acquisition → Service based
- Access → Internet , intranet
- Business → usage based

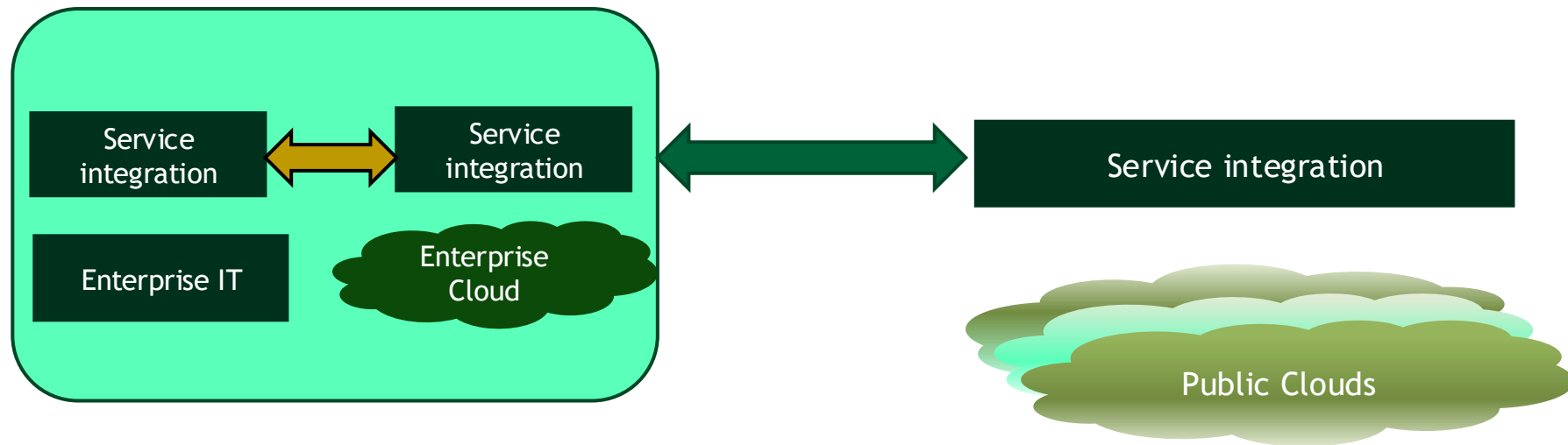
Consumption model

- Self service
- Instance provisioned
- Pay for use
- Scale up and down



Cloud Delivery Models

- **Enterprise** Cloud – sensitive Cloud service delivered behind the firewall
- **Hybrid** Cloud – combination of internal and external cloud services
- **Public** Cloud – general Cloud services delivered over the internet





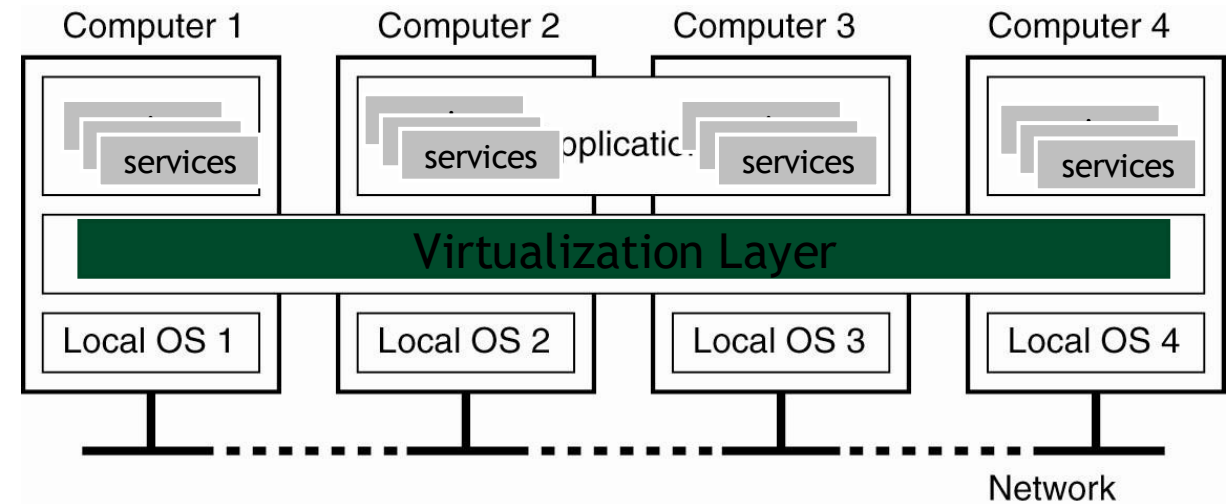
Enabling Cloud

Software

- From Monolithic Applications
- To Distributed services

Infrastructure

- From Loosely connection discrete resources
- To virtualized fabric of resources





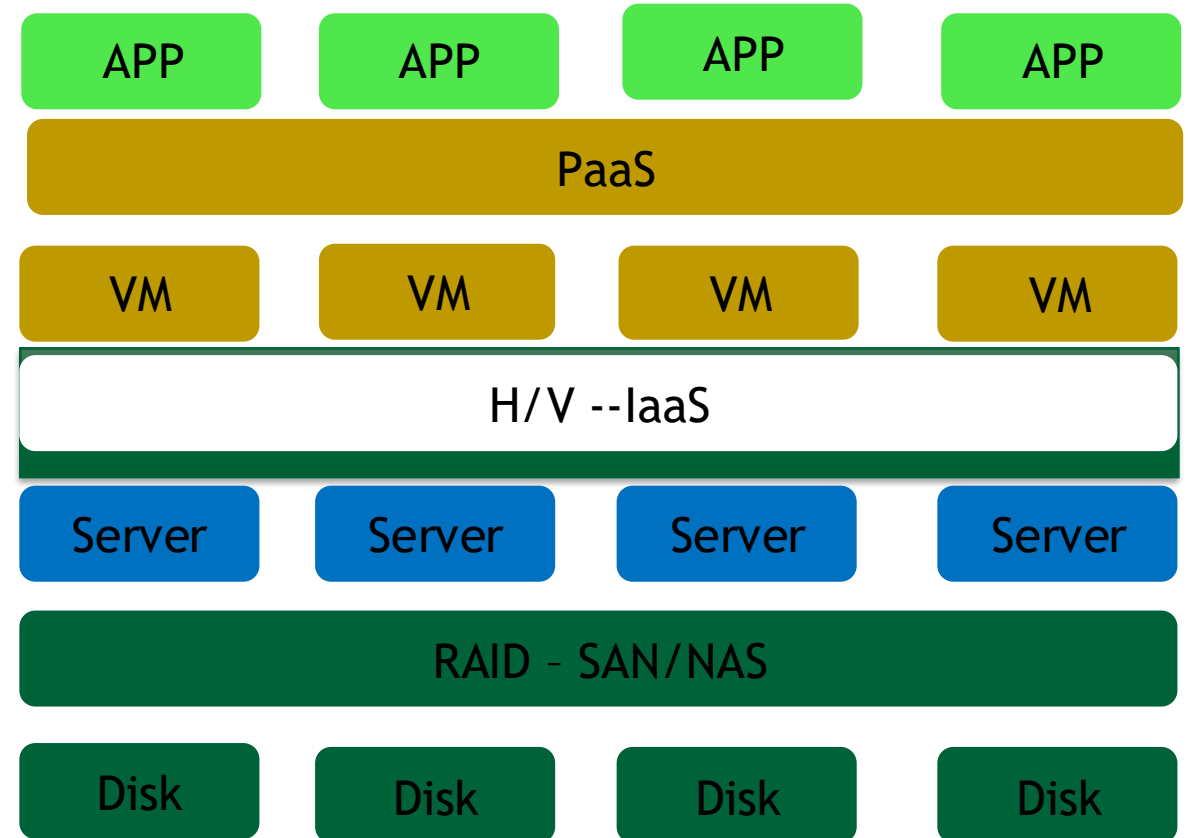
Abstract Pool automate

Software

- From Monolithic Applications
- To Distributed services

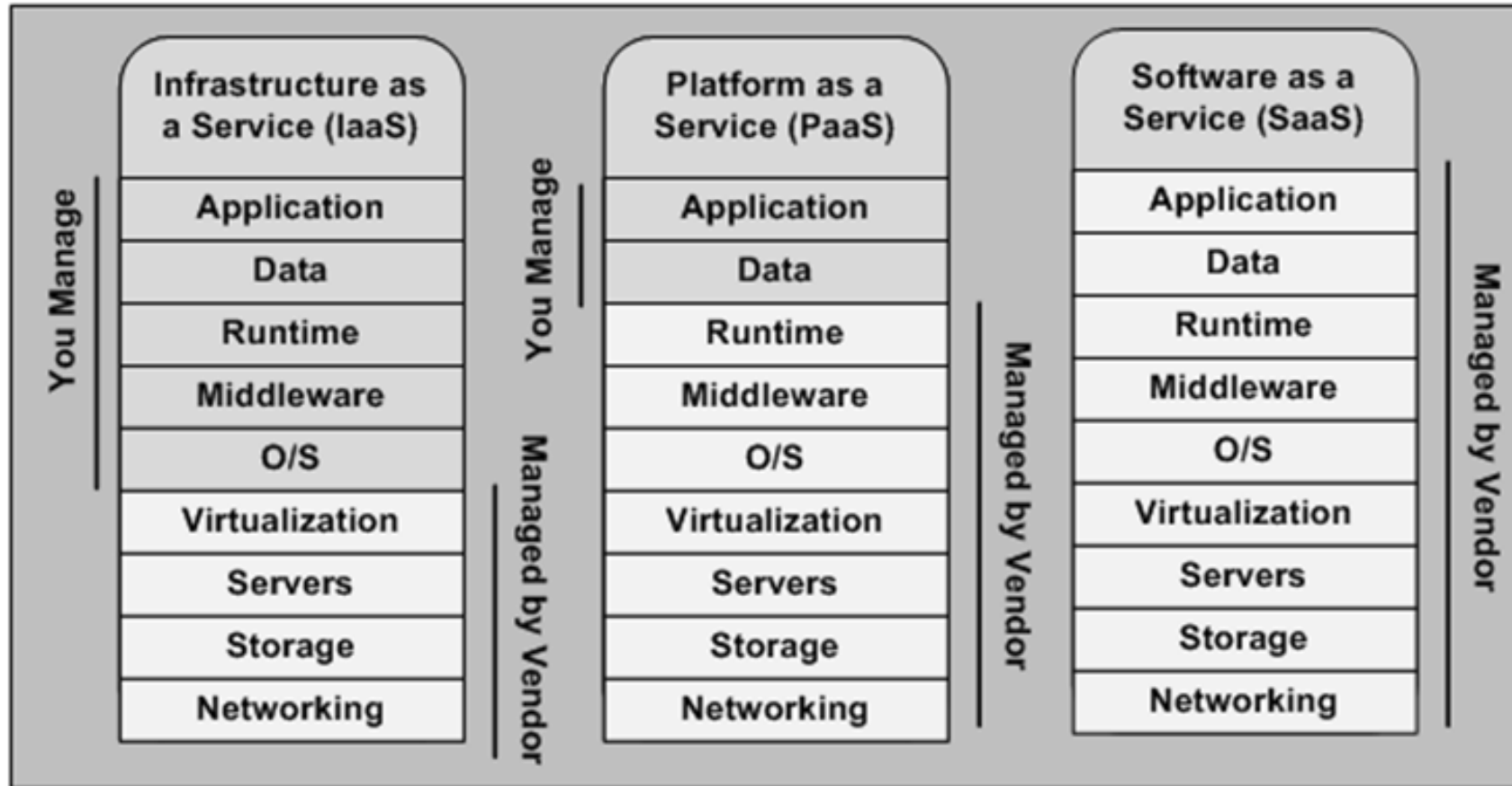
Infrastructure

- From Loosely connection discrete resources
- To virtualized fabric of resources





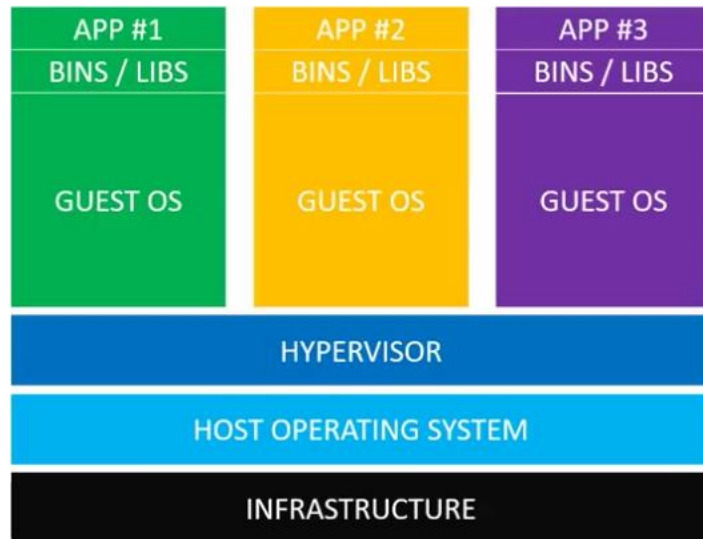
Relation between IaaS, PaaS, SaaS ...



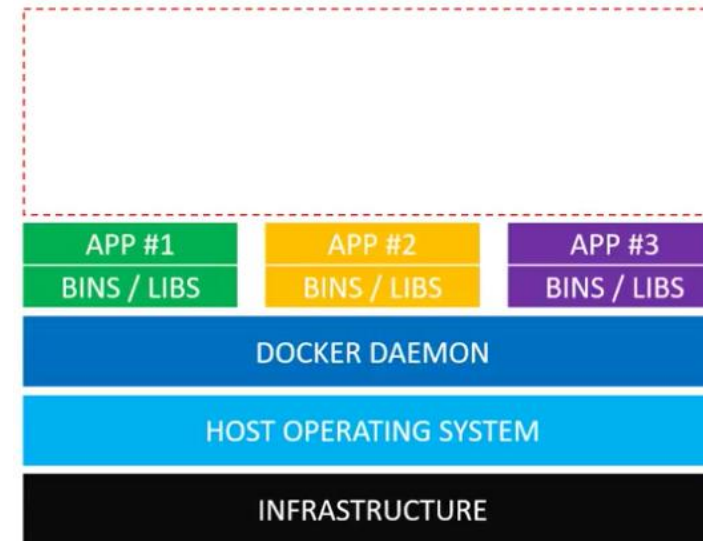


Containers(Dockers) vs VM

- Starts in seconds to a few minutes
 - Isolates systems
 - wastes resources
- Starts in milliseconds
 - Isolates application
 - save resource



Virtual Machines



Docker Containers



Evolution of Container technology

Idea: separate applications from infrastructure

- like how VM separate the OS from bare metal

A Brief History of Containers*: From 1970s **chroot** to Docker 2016

1979: **Chroot** system call introduced during development of Unix Version 7

2000: FreeBSD 4.0 was released with a new command, **called jail**

2004: sun released an early build of Solaris 10, included Solaris **containers** (Solaris zones)

2005: Open VZ created a **hypervisor like** implementation based on **Chroot** and **rlimit**

2006: engineers at Google started the work on this features under “**process containers**”

2007: HP released Secure Resource Partition for HP-UX, Later renamed to **HP-UX containers**,

2008: **LXC (Linux containers) became the first Cgroups based framework to support containers,**

LXC provided a middleware component that allowed the idea of the containers to exist

2013: Docker was release. Started as an internal project within dotCloud. Was build on **LXC** but shifted to **lib container**

Google

IBM

...



Containers

Containers are just **isolated groups of processe(s)** running on a single host.

That isolation leverages several underlying technologies built into the Linux kernel:

- **namespaces**
- **cgroups**

Prominent advantages of containers

- Flexibility
- Convenience
- Consistent
- Reproducibility

Source: Eric Chiang “Containers from Scratch”

<https://ericchiang.github.io/post/containers-from-scratch/>

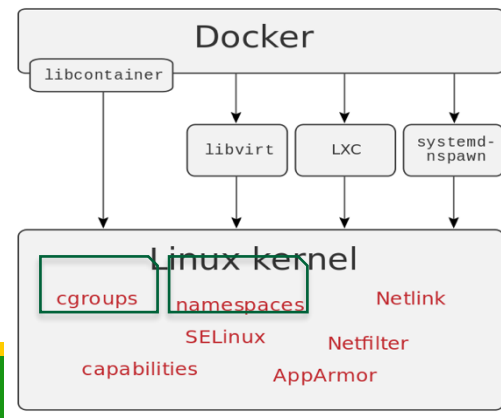
Linux Kernel Support

cgroups: limit how much resource the process can use

- CPU
- Memory
- Network

namespaces: limit what the process can see

- PID
- Net
- mnt
- ipc



Feature	Category
PID (Process ID namespace)	Process Management
UID (User ID namespace)	User & Permission Management
Net (Network namespace)	Networking Subsystem
MNT (Mount namespace)	Filesystem & Storage Management
IPC (Inter-Process Communication namespace)	Process Communication
UTS (UNIX Time-Sharing namespace)	System Identity & Hostname



Container vs Container-image

CONTAINER-IMAGE

Binary representation of the container stored on disk

Images can be **created/built** from

- “*configuration file*” (dockerfile)
- but also from running containers (save container instance)

CONTAINER

A running instance of Container-Image

Packaged with its dependencies

- Nothing is installed on the host
- When the container stops host go to clean state

The **lifecycle of a process** running into the container is tight to the lifecycle of the container process state



Terminology

CONTAINER IMAGE

Persisted snapshot that can be run

`images`: list all local images

`run`: create a container from an image & execute a command in it

`tag`: tag an image

`pull`: download image from repository

`rmi`: Delete a local image. This will also remove intermediate images if no longer used

CONTAINER

Runnable instance of an image

`ps`: List all running containers

`ps -a`: List all container (incl. stopped)

`top`: Display processes of a container

`start`: Start a stopped container

`stop`: Stop a running container

`pause`: Pause all processes within a container

`rm`: Delete a container

`commit`: Create an image from a container

Dockerfile^[1]

A Docker image consists of read-only layers each of which represents a Dockerfile instruction

```
1. FROM ubuntu:18.04
2. COPY . /app
3. RUN make /app
4. CMD python /app/app.py
```

Each instruction creates one layer:

instructions	description
FROM	creates a layer from the ubuntu:18.04 Docker image.
COPY	adds files from your Docker client's current directory
RUN	builds your application with make.
CMD	specifies what command to run within the container.

docker **build** [OPTIONS] PATH | URL | -

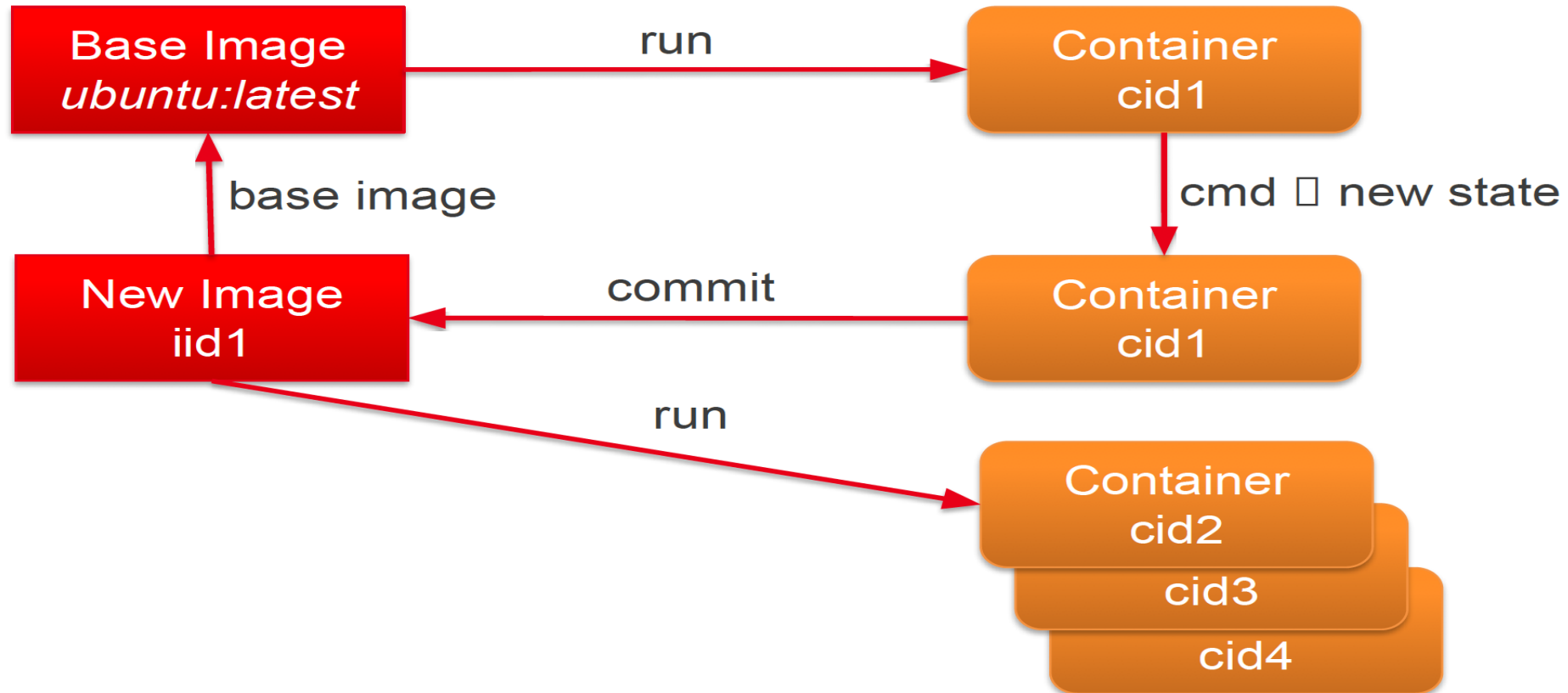
[2]

^[1] Docker reference <https://docs.docker.com/engine/reference/builder/>

^[2] <https://docs.docker.com/engine/reference/commandline/build/>



Container vs Container-image





Docker Containers

DESCRIPTION

The Platform:

The world's leading tool for Application Containerization

The Blueprint (Dockerfile):

A simple text file with a list of instructions used to build the environment automatically

The "Package" (Multi-layered Image):

A read-only binary object containing everything the app needs (code, libraries, settings).

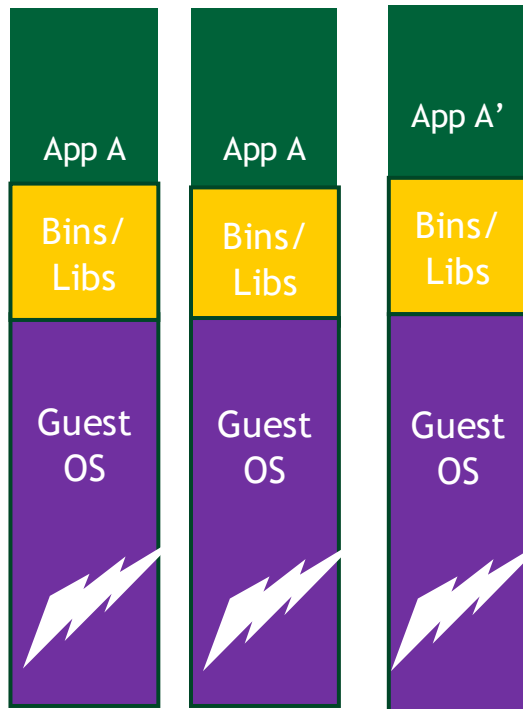
Layered: Each instruction in the Dockerfile creates a new layer, making it extremely storage-efficient.





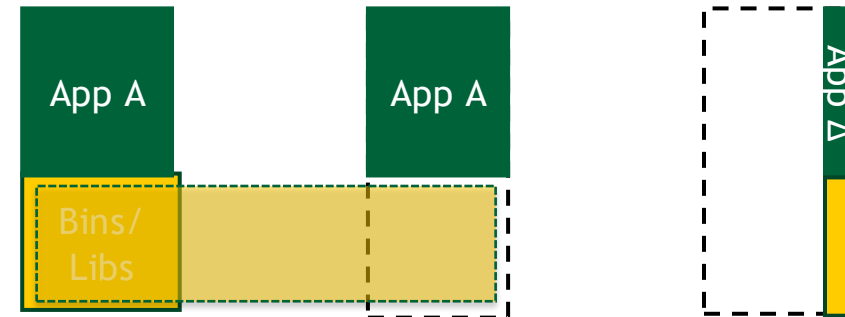
Benefits of layer: copy-on-write

VMs



Every app, every copy of an app, and every slight modification of the app requires a new virtual server

Containers



Original App

Copy of App

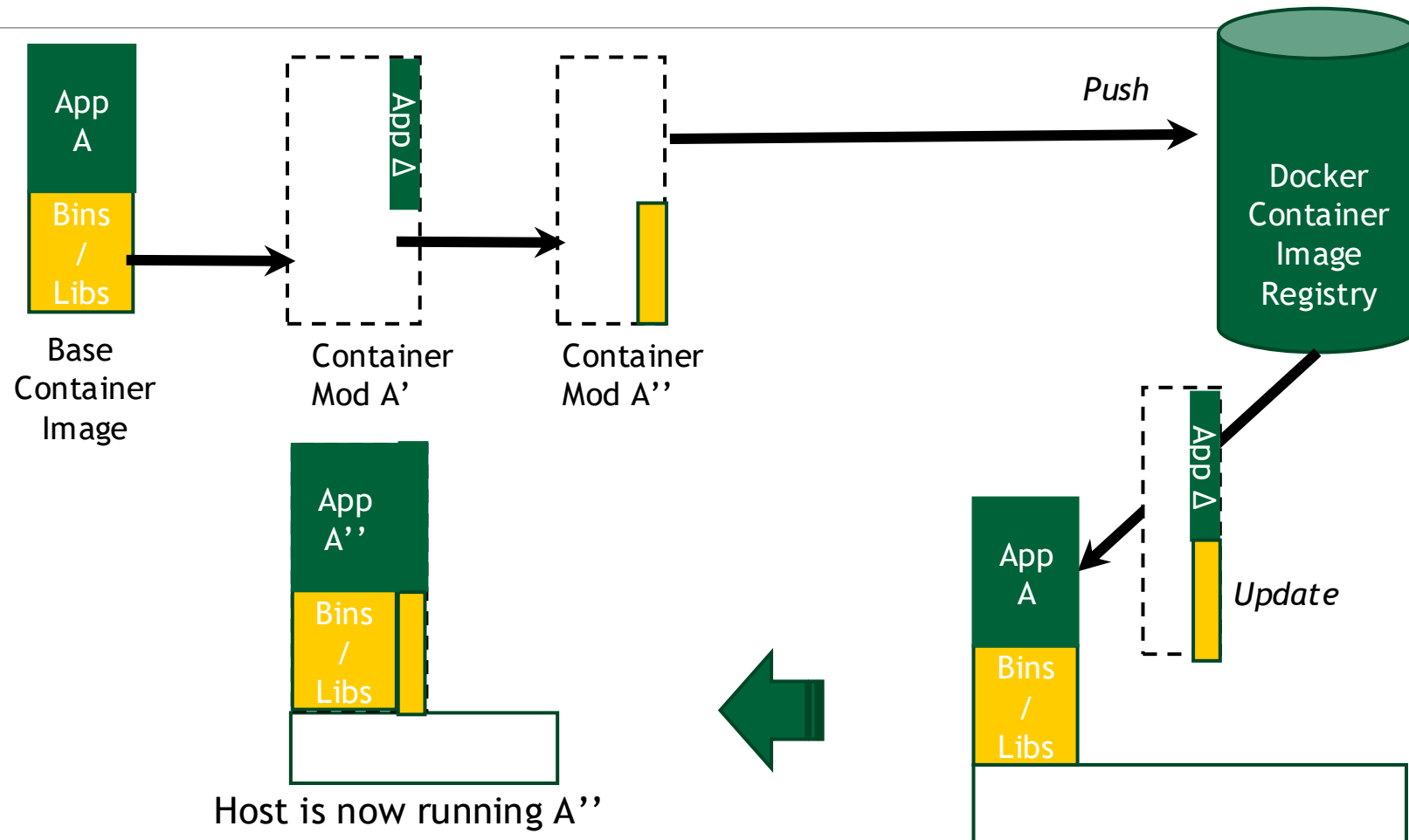
Modified App

(No OS to take up space, resources, or require restart)

No OS. Can **Share** bins/libs

Copy on write capabilities allow us to **only save** the **diffs** Between container A & container A'

Docker container Updates



Host running A wants to upgrade to A''.
Requests update. Gets only diffs



Containers **lifecycle** management tools

1. Discovery and Global Configuration Stores
2. **Creation/Termination**
3. Networking Tools
4. **Management - Scheduling, Orchestration**

Reference: The Docker Ecosystem: An Introduction to Common Components

<https://www.digitalocean.com/community/tutorials/the-docker-ecosystem-an-introduction-to-common-components>



What is Kubernetes (K8s)

"Build once, deploy anywhere." K8s abstracts away the underlying hardware so you can manage containerized apps consistently across the entire computing spectrum.

What K8s Manages: The "Workload"

- **The Unit:** Containers (Docker, containerd, or CRI-O).
- **The Orchestration:** Automates deployment, scaling, and self-healing.
- **The Abstraction:** It treats 100 containers as a single service.

Where it Runs: The "Infrastructure"

Environment	Hardware Context	Key Benefit
Physical	Bare Metal Servers	Raw performance & low latency.
Virtual	Private Clouds (VMware/KVM)	Flexibility & isolated resources.
Cloud	Managed Services (EKS/GKE)	Unlimited scale & hands-off maintenance.



Need for container **orchestration**

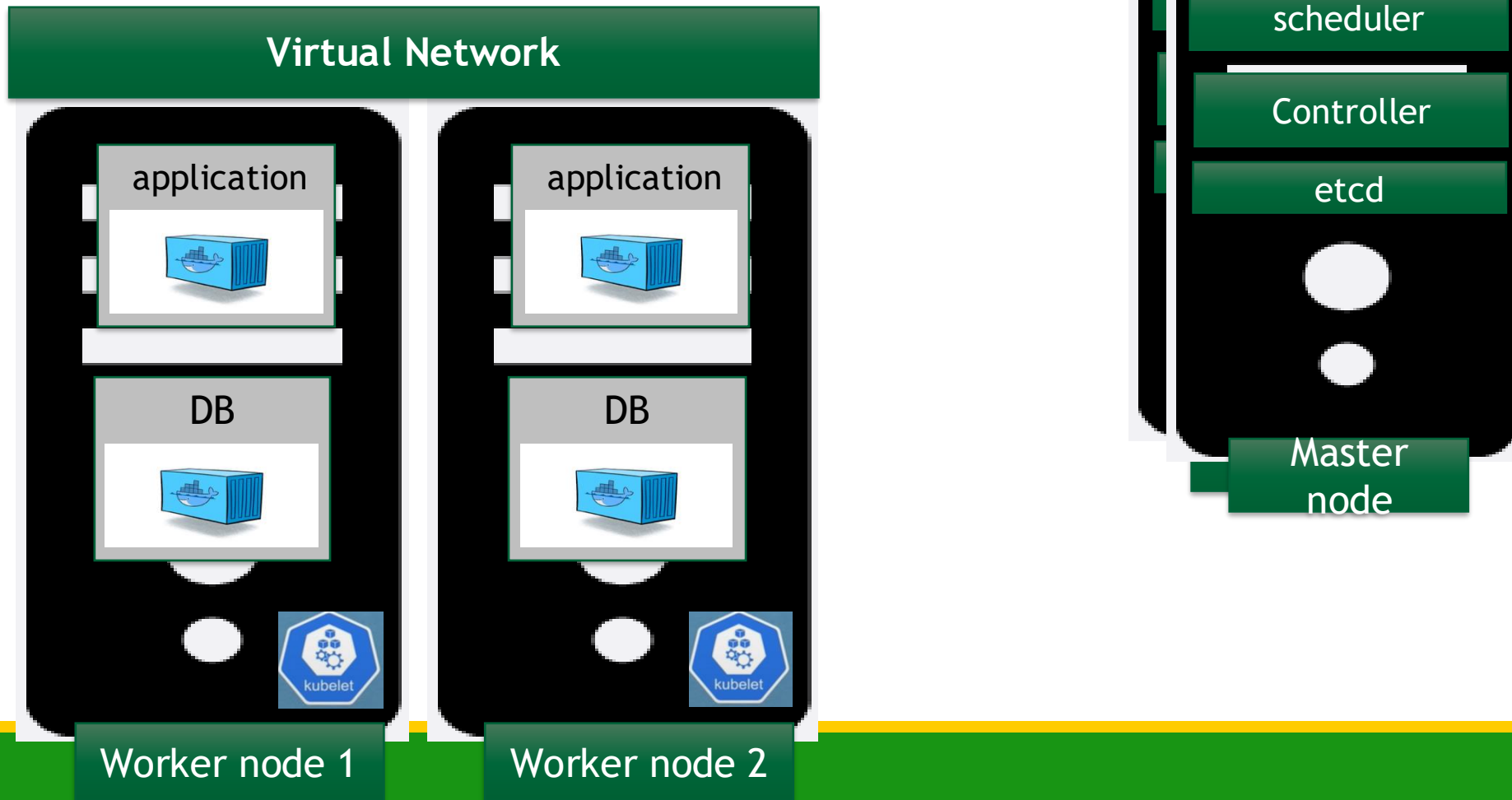
- (1) from monolith to Microservices
- (2) Container Adoption.

Orchestration is the key that unlocks the true power of **microservices at scale**

Core Benefit	What it Achieves
High Availability	Zero Downtime: Redundant infrastructure and automatic failover keep applications online 24/7.
Scalability	Dynamic Performance: Automatically "scale up" or "scale out" based on real-time demand.
Disaster Recovery	Business Continuity: Automated backups and geo-distribution allow for rapid recovery from catastrophic outages.

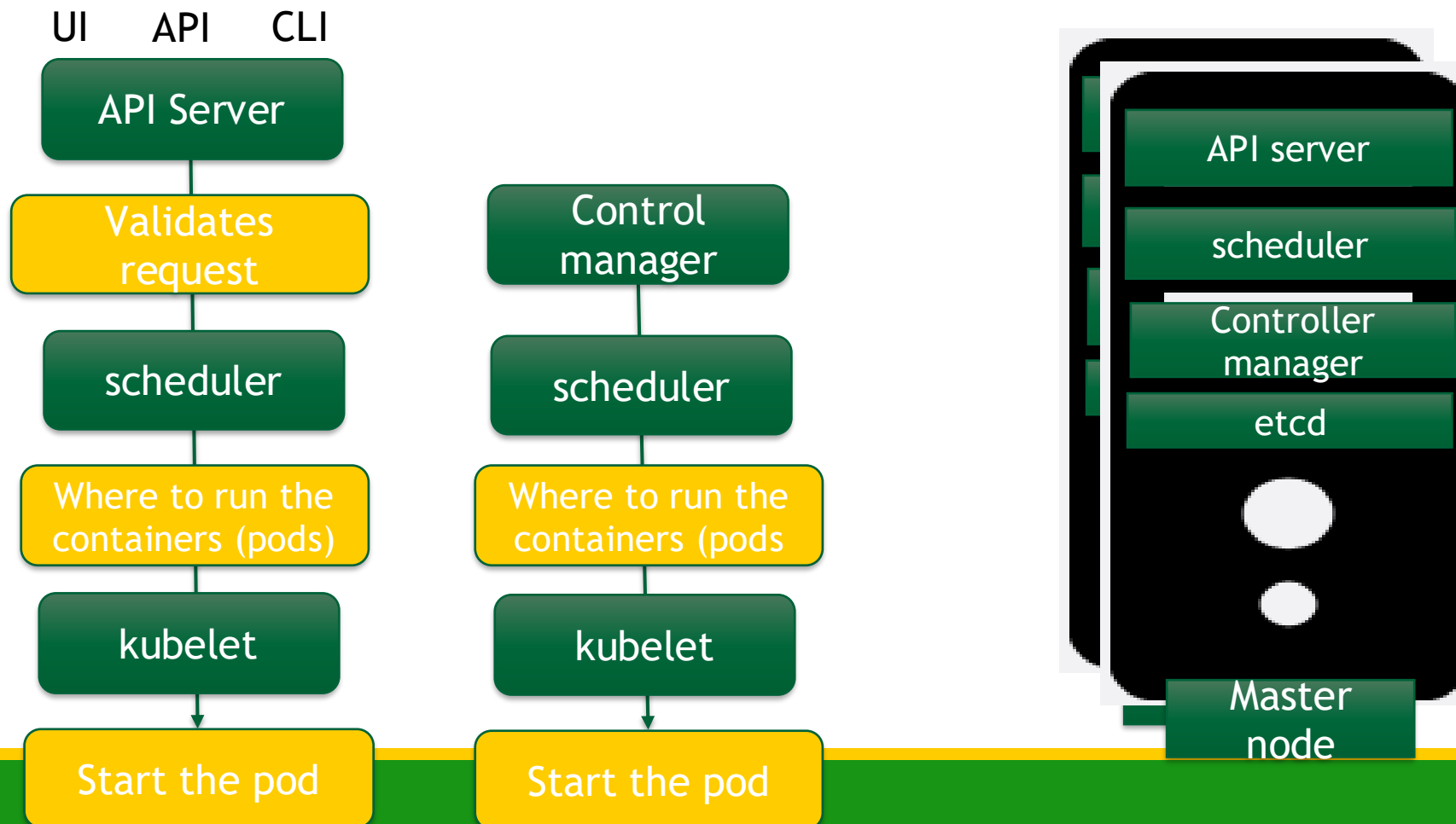


K8s Architecture





Master nodes



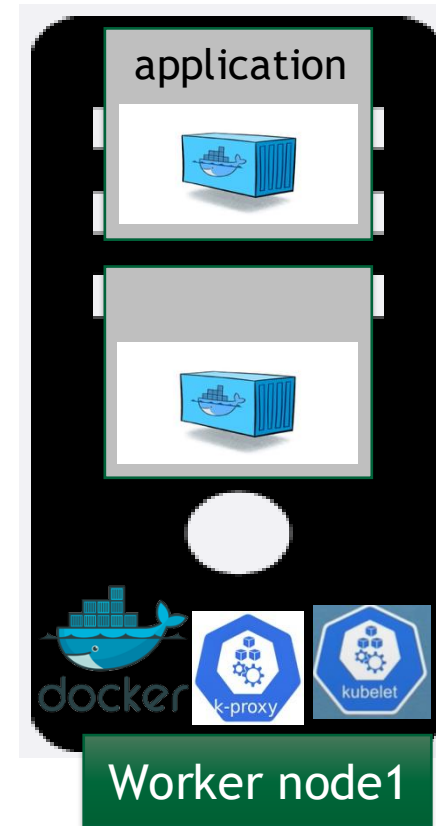


Worker nodes (process nodes)

Support multiple **pods***

must run 3 processes

- Kubelet
- Container runtime
- Kube proxy



* No more container they abstract in a layer called pod



Kubernetes components

Pod

secret

Service

Deployment

Ingress

StatefulSet

ConfigMap

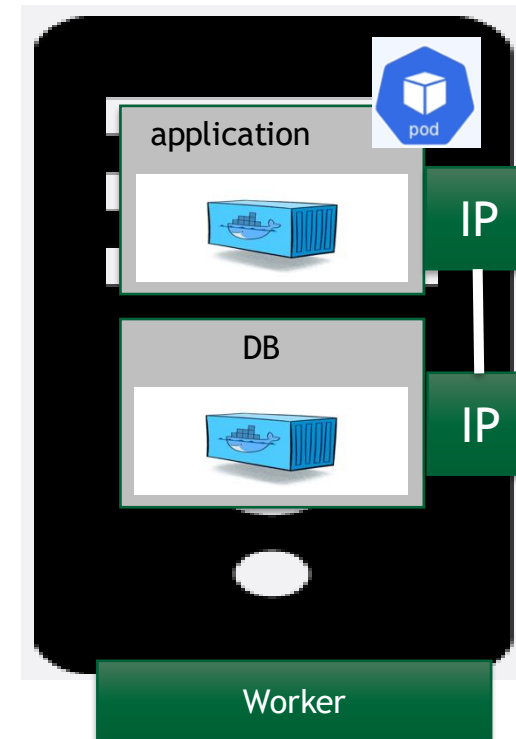
DeamonSet



Kubernetes components

Pods

- **Smallest** unit of K8s
- **Abstraction** over container
- Usually **1 application** per Pod
- Each Pod gets and **IP address**
- New IP address on re-creation



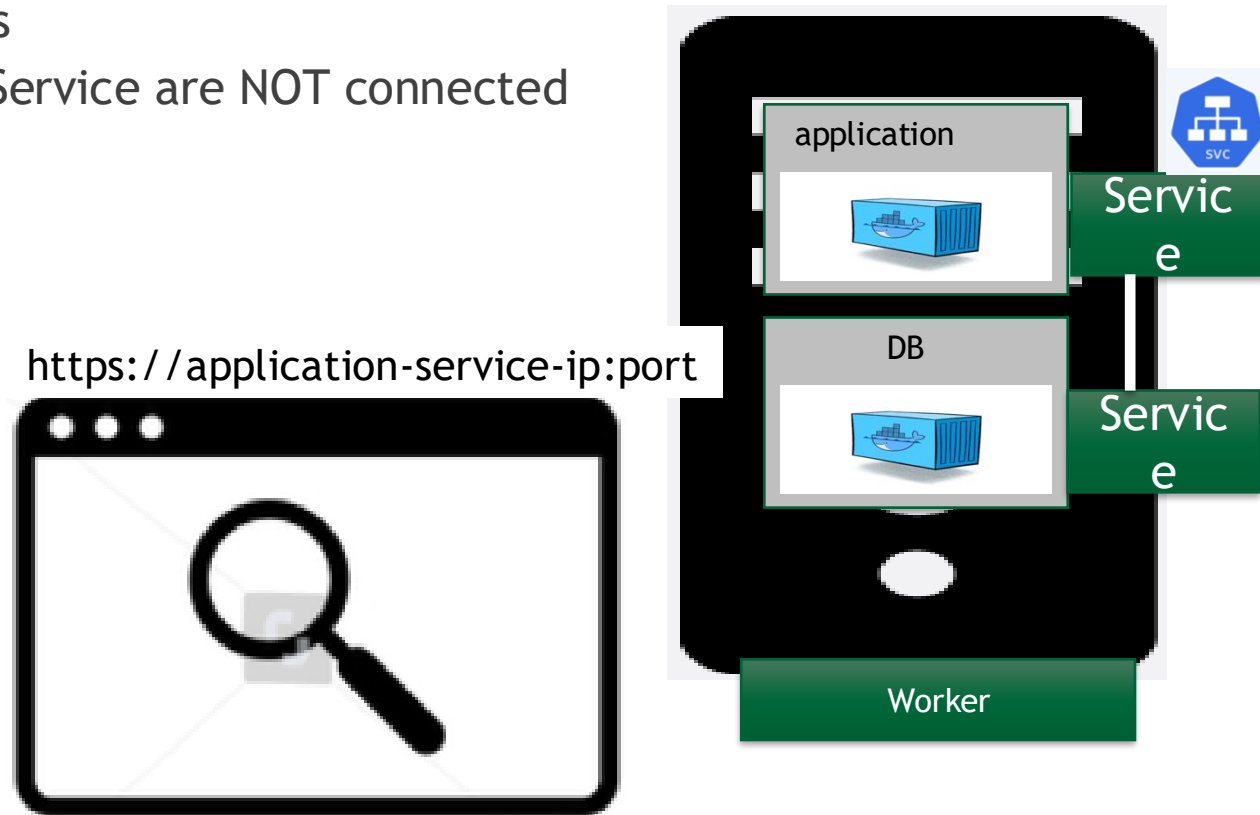
Problem: on restart of failure pods gets new IP?



Kubernetes components

Service

- **permanent** IP address
- lifecycle of Pod and Service are NOT connected



Problem: not all services should be exposed externally



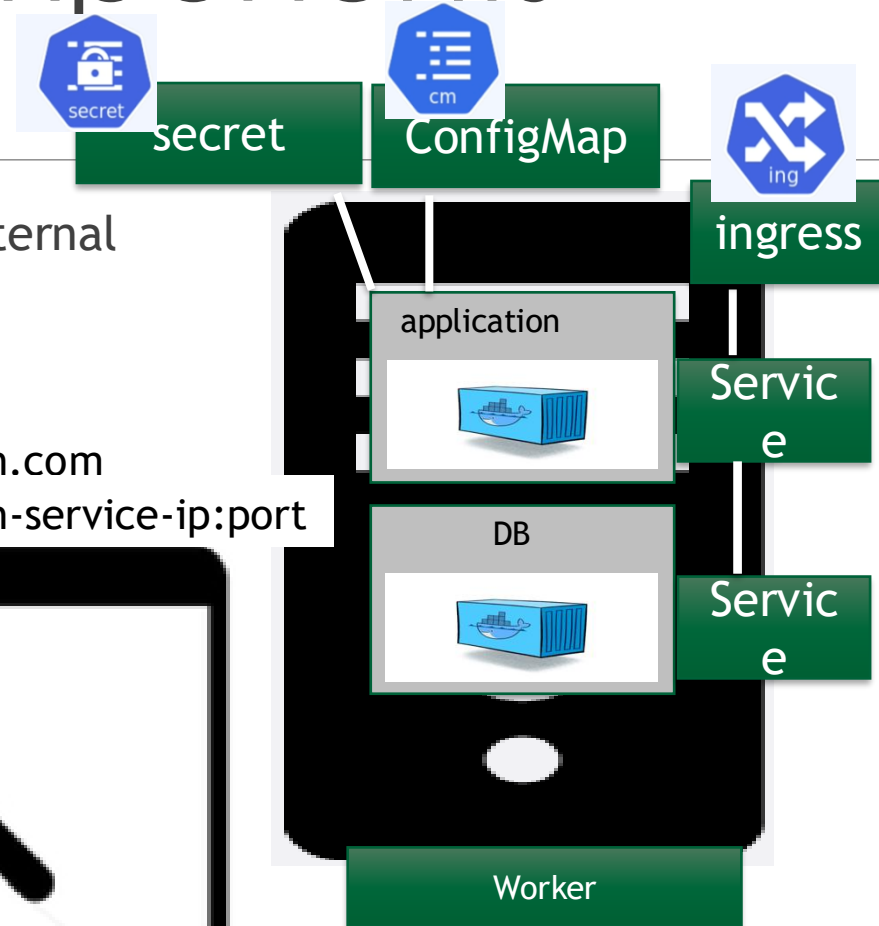
Kubernetes components

Ingress

- opens application to external access (external service)

Secret

<https://application.com>
<https://application-service-ip:port>



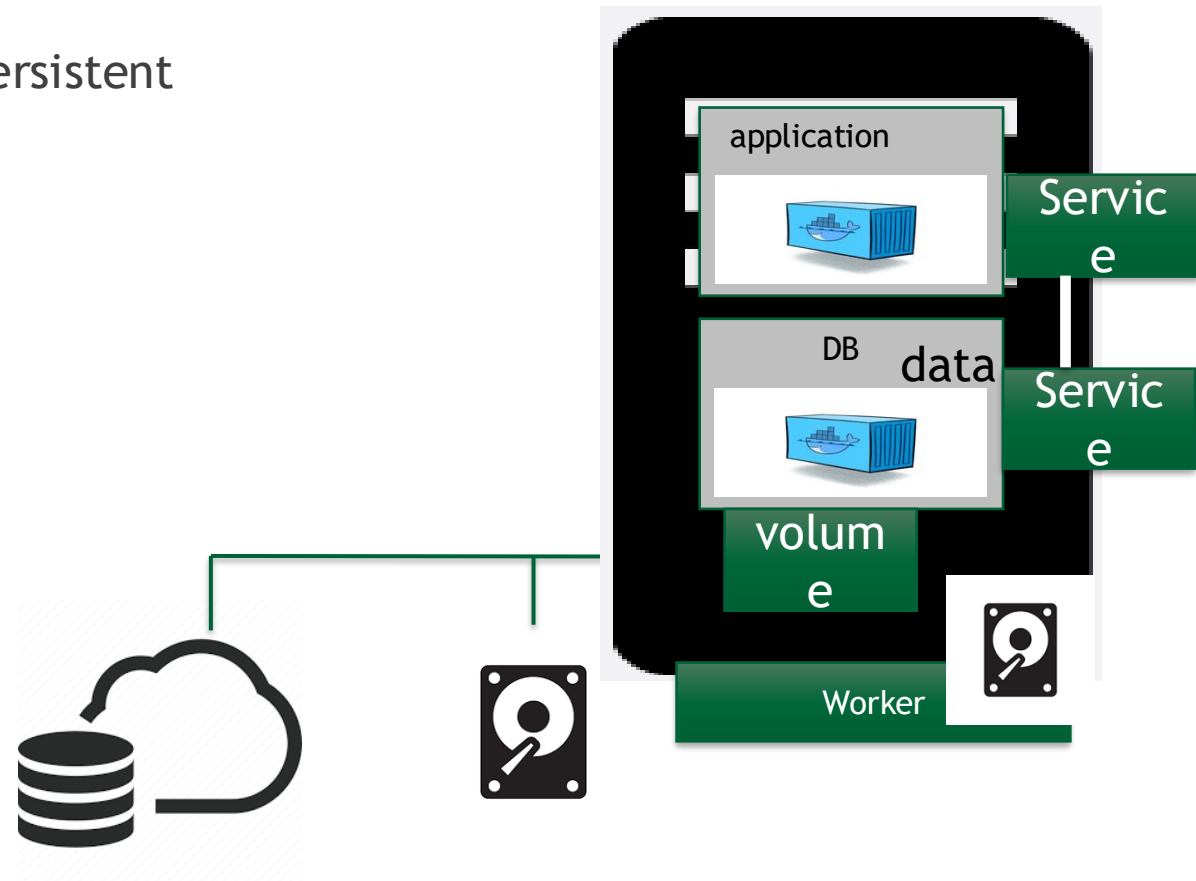
Problem: Data stored inside the pod is not permanent
(if the pod is restarted it is lost) ?



Kubernetes components

Volume

- Permanent storage
- K8s does not manage data persistent



Problem: how to make K8s Robust and scalable?



Deployment

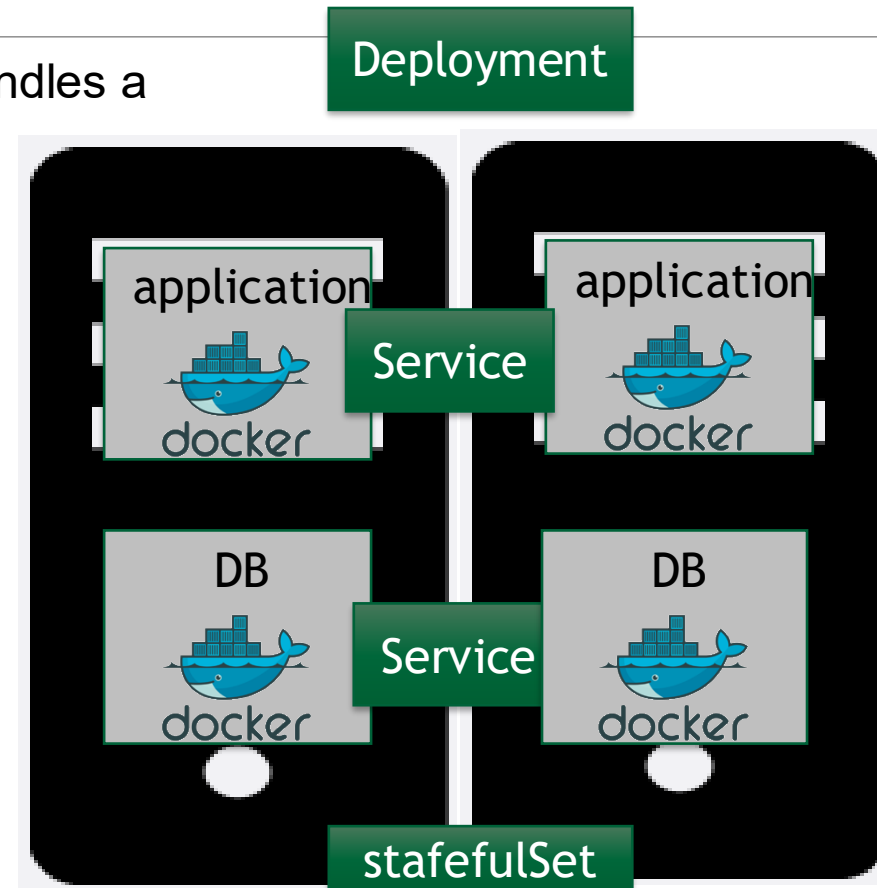
The Goal: You don't care which specific Pod handles a request, as long as one is available.

Deployment

Blueprint for stateless App pods

Key Feature:

- **Self-Healing.** If a Pod dies, the Deployment creates a new one
- **Rolling Updates.** new version of the code, the Deployment replaces Pods one by one - zero downtime.





statefulSet

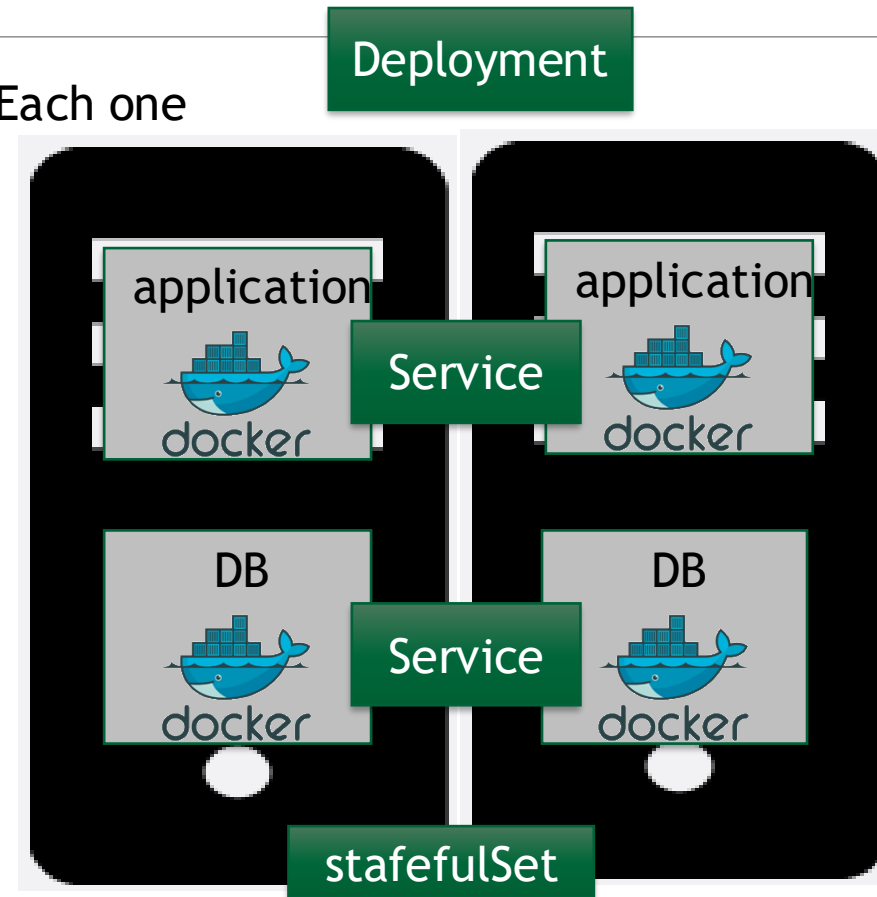
The Goal: The Pods are **not** interchangeable. Each one has a "role" and a permanent memory.

statefulSet

- used for Stateful applications as database

Key Feature:

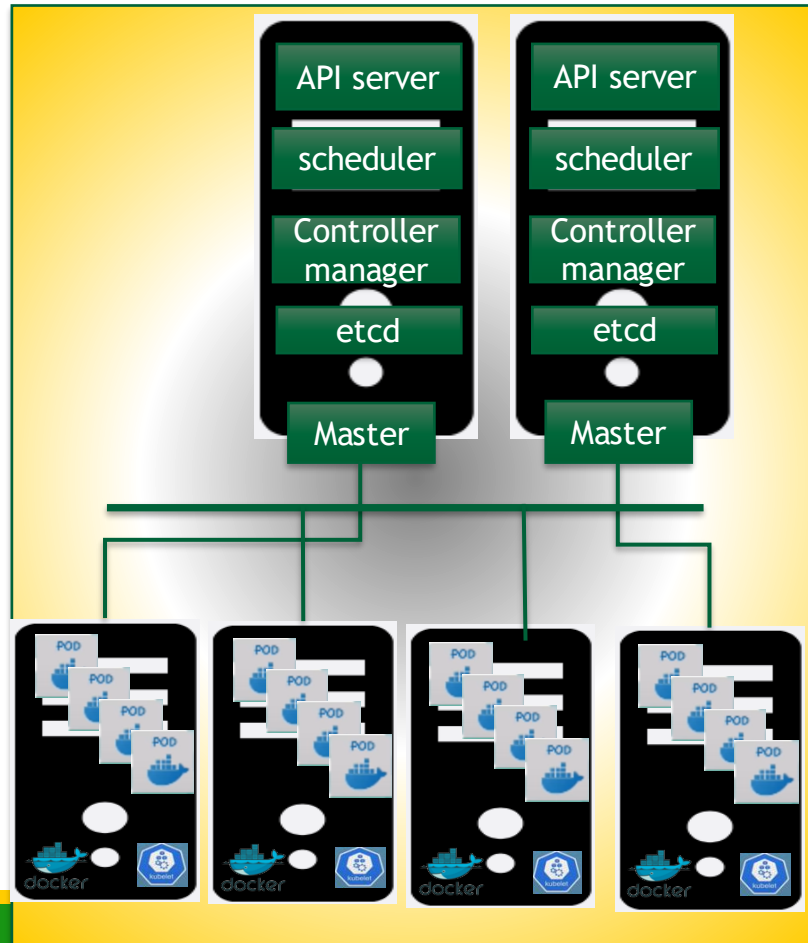
- **Ordered Deployment.** Pods are started in order and shut down in reverse.
- **Stable Storage.** If "Pod-0" crashes and restarts, K8s ensures it re-attaches to the same virtual disk.



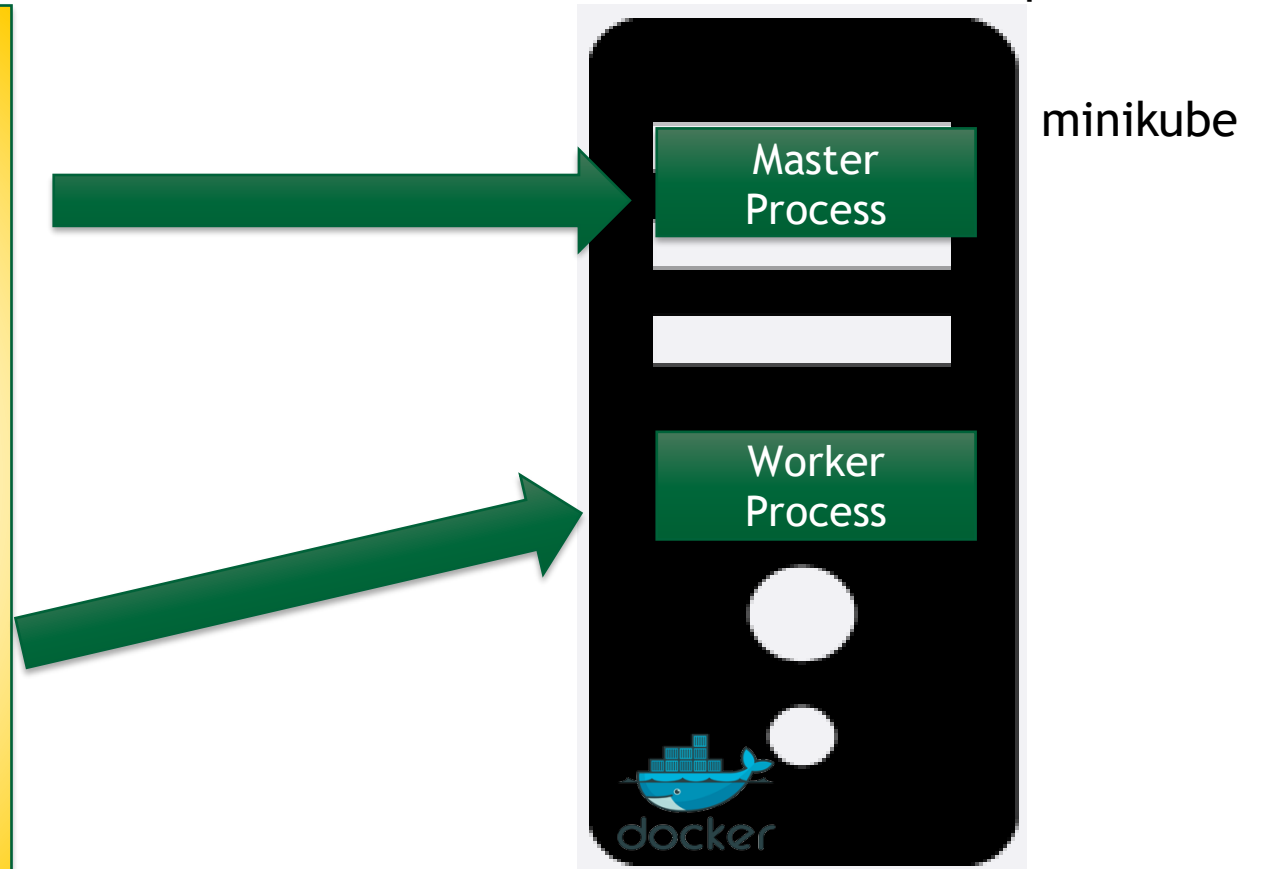


Minikube – K8s tool for a quick local test

Production cluster Setup



Test/local cluster setup

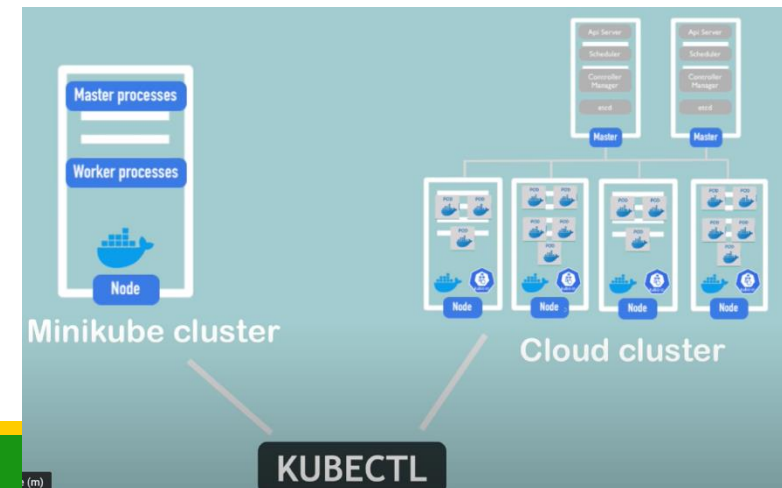




Kubectl

CLI tool used to communicate with the Kubernetes API

- **Deploy**: push App as Pods to the cluster
- **Manage**: Scale and updates
- **Inspect**: pods, services, ...
- **Troubleshoot**



On behalf of the DIGITAfrica consortium



Thank you!